

M16 / PROTECTED ANSWERS

M16 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 4

1. DE16-01 - Production mindset 4

2. DE16-02 - Testing strategy for data engineering 4

3. DE16-03 - Automated quality gates 4

4. DE16-04 - Continuous Integration fundamentals 4

5. DE16-05 - GitHub Actions for data projects 5

6. DE16-06 - Dev, test and prod environments 5

7. DE16-07 - Deployment and rollback 5

8. DE16-08 - Secrets and credential safety 5

9. DE16-09 - RBAC and least privilege 5

10. DE16-10 - PII, encryption and masking 6

11. DE16-11 - Metadata, lineage and governance 6

12. DE16-12 - Observability for data pipelines 6

13. DE16-13 - Incidents, runbooks and root cause 6

14. DE16-14 - Terraform and IaC foundations 6

M16 | WORKED VALIDATION APPENDIX 7

A01 - DE16-01 | Production mindset 7

A02 - DE16-02 | Testing strategy 7

A03 - DE16-03 | Automated quality gates 7

A04 - DE16-04 | Continuous Integration 8

A05 - DE16-05 | GitHub Actions 8

A06 - DE16-06 Dev/test/prod environments	9
A07 - DE16-07 Deployment and rollback	9
A08 - DE16-08 Secrets and credential safety	9
A09 - DE16-09 RBAC and least privilege	10
A10 - DE16-10 PII, encryption and masking	10
A11 - DE16-11 Metadata, lineage and governance	11
A12 - DE16-12 Observability	11
A13 - DE16-13 Incidents/runbooks/RCA	12
A14 - DE16-14 Terraform/IaC	12

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

OPEN ONLY AFTER A SAVED CLOSED-BOOK ATTEMPT

1. DE16-01 - Production mindset

Minimum strong answer: You should finish with a small production contract where another engineer can tell what healthy means, where to look for evidence, and which failure should stop publication. Explain why 'the DAG is green' is not enough to call a data product healthy. Give two examples of successful execution with wrong business output.

Changed retry: Change the case to a streaming payments feed. Decide which promises need tighter latency, what replay/deduplication evidence is required, and what can remain eventually consistent.

2. DE16-02 - Testing strategy for data engineering

Minimum strong answer: A test matrix that names the test layer, fixture, command, expected result, runtime/cost, and defect class. Fast tests belong on every pull request; expensive environment checks belong at stronger release boundaries. Using one orders example, distinguish a unit test, a data-quality expectation, an integration test and a reconciliation check.

Changed retry: The source adds nullable coupon_code. Decide what should continue to pass, what would fail if the field became required, and how backwards compatibility is demonstrated.

3. DE16-03 - Automated quality gates

Minimum strong answer: A result containing rule name, measured value, threshold, severity and final decision. On a hard failure the committed accepted checkpoint must remain unchanged. Why is a validation report alone not a quality gate? Name the additional control that makes it a gate.

Changed retry: Change the volume baseline by 20% and show why the same incoming batch changes decision only because the contractual threshold changed. Record the rule version.

4. DE16-04 - Continuous Integration fundamentals

Minimum strong answer: Every required check either passes or stops the workflow with a non-zero decision. A reviewer can see which gate failed without rerunning your laptop setup. Explain CI to someone who believes it means 'automatically deploy every commit.' What is missing from that definition?

Changed retry: First make the unit-test stage fail and prove deploy is unreachable. Then fix it and introduce a secret-pattern failure so a different gate blocks the path.

5. DE16-05 - GitHub Actions for data projects

Minimum strong answer: You can identify trigger, job, runner image, reusable actions, shell commands, permissions and fail-closed checks. You can also explain which steps could run safely on an untrusted pull request. What is the difference between `uses:` and `run:`? Why should workflow permissions be designed explicitly?

Changed retry: Add a second job that depends on `validate` and produces a harmless text artifact. Break `validate` and prove the artifact job does not run; then repair it.

6. DE16-06 - Dev, test and prod environments

Minimum strong answer: You can show the same artifact/version across environments while endpoints, identities and capacities change through explicit configuration. No secret value is baked into the artifact. Why should promotion prefer the same artifact with different bindings rather than three manually edited copies?

Changed retry: Swap one test binding to a production resource and prove your checker rejects it. Then add an approved read-only prod reference and explain why its risk differs.

7. DE16-07 - Deployment and rollback

Minimum strong answer: A deployment record naming version, environment, checks, decision, rollback target and affected data window. Rollback restores code safely without pretending data repair is automatic. Why is code rollback not the same as data rollback? Give one example where the old code is restored but the system remains wrong.

Changed retry: Change the case so `v2` writes an idempotent derived table rather than destructive state. Explain why replay is now easier and what evidence still matters.

8. DE16-08 - Secrets and credential safety

Minimum strong answer: You can explain where secret material may leak (code, git history, logs, notebooks, config exports) and what to do after exposure: revoke/rotate, investigate use, remove from future code/history where appropriate, and replace the design. Why does deleting a leaked credential from the latest source file not make the credential safe again?

Changed retry: Assume OIDC is temporarily unavailable for one legacy service. Design a contained secret fallback with rotation, restricted scope, masked logs and an explicit removal date.

9. DE16-09 - RBAC and least privilege

Minimum strong answer: A role matrix where principal, action, scope and justification are explicit. Broad roles have documented exception reasoning rather than being default troubleshooting tools. Explain principal + role + scope with one Azure example. Why can a management-plane role differ from data access?

Changed retry: Change the transformation task so it needs to rotate a secret. Decide whether to expand its existing role or delegate rotation to a separate automation identity.

10. DE16-10 - PII, encryption and masking

Minimum strong answer: You can classify fields, identify raw versus masked zones, state who needs unmasked access, and explain how logs/telemetry avoid recording PII values. Why are encryption and masking different controls? Give one failure that encryption alone does not prevent.

Changed retry: Add a legitimate fraud-use case requiring a small group to see raw phone numbers. Define the narrow access, audit and retention controls rather than weakening every dataset.

11. DE16-11 - Metadata, lineage and governance

Minimum strong answer: A lineage graph that supports both root-cause questions ('where did this come from?') and impact questions ('what will break if I change this?'). Known lineage gaps are stated instead of hidden. What can lineage answer that a schema alone cannot? Give one upstream root-cause and one downstream impact question.

Changed retry: Insert a second consumer and a shortcut/external table whose lineage is only partially captured. Mark the gap and document a manual control until automated lineage is available.

12. DE16-12 - Observability for data pipelines

Minimum strong answer: A compact health record with software and data signals, enough context to route an incident, and no raw PII as high-cardinality telemetry labels. Why can CPU/runtime/success metrics all be healthy while a data product is unhealthy? Name three data-specific signals.

Changed retry: Change the incident so row count is normal but freshness is six hours late. Decide which alert should fire and why a pure volume dashboard would miss it.

13. DE16-13 - Incidents, runbooks and root cause

Minimum strong answer: A timeline separating observed facts from hypotheses, containment action, recovery proof, customer/data impact, causal chain and prevention tasks with owners. What is the difference between a runbook and an RCA? Why should facts and hypotheses be labeled separately during an incident?

Changed retry: Change the symptom to high latency with correct output. Decide which containment and evidence steps differ and which remain the same.

14. DE16-14 - Terraform and IaC foundations

Minimum strong answer: You can explain what config, provider, state, plan and apply each do; you do not call a successful plan proof that the data pipeline itself is correct. Explain why terraform plan is valuable but does not prove application or data correctness. What extra evidence belongs after infrastructure deployment?

Changed retry: Change one harmless local variable in the sample configuration and predict the plan effect. Then imagine the same change affects a production storage account; list additional approvals and data-safety checks.

M16 | WORKED VALIDATION APPENDIX

Use this appendix only after a saved attempt. It turns short validation points into concrete solved examples, failure checks and changed retries.

A01 - DE16-01 | Production mindset

Concrete solved example

Concrete production contract: hourly orders pipeline must publish by hh:15; source_count = accepted + quarantine; serving order_id unique; paid_revenue reconciles to accepted paid rows; checkpoint advances only after publish; owner/on-call/runbook are named.

Worked proof / evidence

Green DAG but wrong result examples: filter drops yesterday orders; join fan-out doubles revenue while tasks all succeed.

Common mistake

Equating orchestration success with data-product health.

Validation

Review SLO, data invariants, evidence location, owner and stop-publish conditions.

Changed retry

Change to streaming payments and tighten freshness/replay/dedup promises.

A02 - DE16-02 | Testing strategy

Concrete solved example

Example matrix: unit test net_sales function with tiny fixture on every PR; data-quality test order_id uniqueness on every build; integration test local DB/API path before merge/release; reconciliation source vs target counts/revenue at runtime.

Worked proof / evidence

Unit: pure function. DQ: dataset property. Integration: components talk. Reconciliation: end-to-end population/business equality.

Common mistake

Running only unit tests and calling the pipeline production-tested.

Validation

For each test record fixture, command, expected result, runtime/cost and defect class.

Changed retry

Add nullable coupon_code and predict which tests should remain green.

A03 - DE16-03 | Automated quality gates

Concrete solved example

A real gate has measured value, threshold/severity and a fail-closed decision wired to publication. Example: `duplicate_keys=2, threshold=0, severity=hard -> publish blocked; committed checkpoint unchanged.`

Worked proof / evidence

if `duplicates>0`: `exit(2)` # CI/runtime gate stops next step

Common mistake

Generating a validation report but continuing publish regardless.

Validation

Break one rule and prove downstream publish/deploy job is unreachable and state unchanged.

Changed retry

Change threshold intentionally with versioned rule and predict decision.

A04 - DE16-04 | Continuous Integration

Concrete solved example

CI validates integrated changes automatically; it is not “deploy every commit.” A PR workflow can run lint/unit/contract/secret scan and block merge on nonzero failure; deployment is a separate controlled stage.

Worked proof / evidence

PR -> checkout -> install -> tests -> validation -> secret scan -> PASS/FAIL

Common mistake

Treating CI and CD as synonyms; ignoring failed command exit codes.

Validation

Force unit test failure and prove later deploy/release step is not executed.

Changed retry

Repair unit test then trigger a secret-pattern failure to prove a different gate blocks.

A05 - DE16-05 | GitHub Actions

Concrete solved example

Example workflow uses pinned/reviewed actions, explicit permissions, a runner image and run commands. `uses:` invokes an action/reusable workflow; `run:` executes shell commands. Start permissions read-only and grant only needed scopes.

Worked proof / evidence

```
permissions:
  contents: read
  id-token: write # only job that needs OIDC
```

Common mistake

Using write-all permissions or exposing secrets to untrusted fork PR steps.

Validation

Inspect trigger, permissions, dependencies and failed-job propagation in a test PR.

Changed retry

Add artifact job needs: validate; break validate and prove artifact job is skipped.

A06 - DE16-06 | Dev/test/prod environments

Concrete solved example

Promote the same versioned artifact while binding DEV/TEST/PROD endpoints, identities, capacities and secrets through explicit environment configuration.

Worked proof / evidence

artifact sha abc123 in all envs; bindings differ: lakehouse_dev/test/prod

Common mistake

Maintaining three hand-edited code copies.

Validation

Compare artifact/version hashes and environment binding matrix; scan artifact for secrets.

Changed retry

Point TEST binding at PROD and prove environment guard rejects it.

A07 - DE16-07 | Deployment and rollback

Concrete solved example

Deployment record: version v2, target PROD, checks, approval, data window, rollback target v1. Code rollback restores v1 executable behavior; if v2 already corrupted data, separately restore/recompute affected data.

Worked proof / evidence

deploy v2 -> wrong derived rows -> rollback code v1 + replay/rebuild data window

Common mistake

Assuming code rollback automatically undoes data mutations.

Validation

Test rollback artifact and reconcile affected target data after repair.

Changed retry

Use an idempotent derived table and compare recovery complexity.

A08 - DE16-08 | Secrets and credential safety

Concrete solved example

If a token leaks in git/logs: revoke/rotate first, investigate use, remove value from future code/history as appropriate, replace with OIDC/workload identity/managed secret store, and add prevention. Deleting the latest file does not invalidate copies/history.

Worked proof / evidence

incident evidence: secret id (not value), exposure window, rotation time, affected systems

Common mistake

Committing a new secret before revoking the old one; printing secret during diagnosis.

Validation

Secret scan/log review shows no value; old credential is invalid; new path uses narrow identity.

Changed retry

Design temporary secret fallback with restricted scope/rotation/removal date.

A09 - DE16-09 | RBAC and least privilege

Concrete solved example

Azure example: deployment service principal + Contributor-like deployment role scoped only to DEV resource group/workspace, while data access uses a separate storage/data-plane role scoped to required container/table. Management-plane and data-plane permissions answer different actions.

Worked proof / evidence

principal -> action -> role -> scope -> justification matrix

Common mistake

Granting Owner/Admin because a 403 occurred without identifying denied action/resource.

Validation

Capture principal, denied action, target resource and effective role scope before changing access.

Changed retry

Separate secret rotation into a dedicated automation identity.

A10 - DE16-10 | PII, encryption and masking

Concrete solved example

Encryption protects data at rest/in transit from certain access paths; masking/tokenization changes what authorized viewers/processes can see. An analyst with legitimate decrypt access can still overexpose raw phone numbers unless masking/RBAC/minimization apply.

Worked proof / evidence

raw zone restricted; curated view masks phone -> ***1234; logs carry customer_id hash not phone

Common mistake

Assuming encryption means every authorized query is privacy-safe.

Validation

Classify fields, list raw/masked zones and access roles; inspect logs for PII values.

Changed retry

Grant a fraud team narrowly scoped raw-phone access with audit/retention.

A11 - DE16-11 | Metadata, lineage and governance

Concrete solved example

Schema says what fields exist; lineage answers origin/dependencies. Root-cause: “Which source/run/model produced wrong gold_revenue?” Impact: “Which dashboards/models consume customer_status before we change it?”

Worked proof / evidence

source -> bronze -> silver -> gold -> BI graph with run/model metadata

Common mistake

Calling a table schema “lineage”; hiding manual/external gaps.

Validation

Pick one output and trace upstream to source, then one source field and list downstream dependents.

Changed retry

Add an external shortcut with partial lineage and document manual gap control.

A12 - DE16-12 | Observability

Concrete solved example

CPU/runtime/task success can be healthy while data is stale, duplicated or semantically wrong. Data signals: freshness/age, source-vs-target counts, duplicate/quarantine rate, business metric reconciliation, schema drift.

Worked proof / evidence

health record={run_id,status,duration,freshness_sec,source_rows,target_rows,quarantine,paid_revenue}

Common mistake

Only infrastructure metrics; using raw customer email as telemetry label.

Validation

Inject six-hour stale input with normal row count and prove freshness alert fires.

Changed retry

Keep volume normal but introduce duplicate keys and identify the right alert.

A13 - DE16-13 | Incidents/runbooks/RCA

Concrete solved example

Runbook is prewritten operational procedure (“if lag grows, check X/Y, contain, recover”). RCA is post-incident causal analysis with timeline, impact, contributing causes and prevention. During incident, label observed facts separately from hypotheses to avoid turning guesses into actions.

Worked proof / evidence

FACT 02:10 source rows=0; HYPOTHESIS upstream API outage; FACT 02:14 API 200 but empty payload

Common mistake

Writing RCA while incident is active and presenting guesses as facts.

Validation

Timeline includes evidence timestamps, containment, recovery proof and prevention owners.

Changed retry

Change symptom to latency-only with correct output and revise containment/evidence.

A14 - DE16-14 | Terraform/IaC

Concrete solved example

terraform plan shows proposed infrastructure diff before apply; it does not prove provider/API side effects will succeed or that the deployed data pipeline processes correct data. After deploy, run infrastructure smoke checks plus application/data validation.

Worked proof / evidence

```
terraform fmt -check
terraform validate
terraform plan
# after apply: identity/network/storage smoke + pipeline test fixture + reconciliation
```

Common mistake

Calling plan success production validation or storing sensitive state insecurely.

Validation

Compare plan to approved change, apply in safe scope, then run target connectivity and known-data smoke test.

Changed retry

Change a production storage setting and list extra approval/state/backup/data-safety checks.